

Toxic Comment Detection Using MLP and Bidirectional LSTM

Course Project Report

1. Problem

Online platforms like Wikipedia, Reddit, and various social media sites deal with a huge amount of user-generated content every day. A big chunk of that content includes toxic comments such as insults, threats, hate speech, and obscenity. Moderating all of this manually is not possible because the volume is just too high. So, there is a real need for systems that can automatically detect whether a comment is toxic or not.

This project focuses on building such a system. We treat this as a binary text classification task where the model takes a comment as input and predicts whether it is toxic (label 1) or not toxic (label 0).

Two different models were built and compared:

- A **TF-IDF + MLP** model, which is a more traditional machine learning approach where we first extract features from text using Term Frequency-Inverse Document Frequency (TF-IDF), and then pass those features through a small neural network.
- A **Bidirectional LSTM** model, which is a deep learning approach that reads the text word by word and understands the context and order of words.

The dataset used for this project is the **Jigsaw Toxic Comment Classification Challenge** dataset from Kaggle. It contains real comments from Wikipedia talk pages, each labeled as toxic or non-toxic. The test set has 63,978 samples.

One important challenge with this dataset is class imbalance. Most comments in the dataset are non-toxic, which means a model could simply predict "not toxic" every single time and still achieve high accuracy. Because of this, accuracy alone is not a good measure. We focused more on F1-score, precision, recall, and ROC-AUC to properly evaluate how well the models work.

2. Approach

2.1 Data Preprocessing

Before training any model, the raw text needs to be cleaned. A few simple steps were applied:

- Converting everything to lowercase
- Removing URLs
- Replacing numbers with a placeholder token "NUM"

- Reducing repeated characters (for example, "heeellooo" becomes "heelo")
- Removing punctuation and special characters

These steps help both models see cleaner, more consistent input.

Since the dataset was heavily imbalanced, we applied **undersampling** on the majority class (non-toxic comments) to bring the class distribution closer to balance before training the TF-IDF model.

2.2 Model 1: TF-IDF + MLP (Scikit-Learn)

The first model takes a more traditional approach. The text is converted into a numerical feature vector using TF-IDF. We used up to 50,000 features including both single words (unigrams) and two-word combinations (bigrams).

These feature vectors are then passed into a Multi-Layer Perceptron (MLP) with 3 dense hidden layers, built using Scikit-Learn's **MLPClassifier**.

The key hyperparameters used:

- Max features: 50,000 (unigrams + bigrams)
- Undersampling to balance classes before training
- Early stopping based on validation score with a tolerance of 0.0001
- Classification threshold was not fixed at 0.5. Instead, it was tuned separately.

2.3 Model 2: Bidirectional LSTM (PyTorch)

The second model is a proper deep learning model. Each word in the comment is first converted to a dense vector using an **Embedding layer** (embedding dimension = 100). These embeddings are then fed into a **Bidirectional LSTM** layer with a hidden size of 128. Because it is bidirectional, it reads the sentence both forward and backward, so it captures better context.

After the LSTM, we apply **Dropout** (rate = 0.3) to reduce overfitting. The final hidden states from both directions are concatenated and passed to a fully connected layer that outputs a single logit. A sigmoid function converts this to a probability between 0 and 1.

The architecture in brief:

```
Input (text) -> Embedding Layer (vocab_size x 100)
              -> Bidirectional LSTM (hidden=128, both directions)
              -> Dropout (0.3)
              -> Fully Connected Layer (256 -> 1)
              -> Sigmoid -> Probability
```

The model was trained for 8 epochs with binary cross-entropy loss.

2.4 Threshold Tuning

One thing both models have in common is that we did not use the default classification threshold of 0.5. Instead, we used a two-phase training approach:

1. First, train the model on 80% of the data and keep 20% as a validation split.
2. On the validation set, compute the precision-recall curve and find the threshold that gives the highest F1-score.
3. Then retrain the model on the full training data and apply the selected threshold during testing.

This approach helps in dealing with imbalanced data because the right threshold can be picked based on actual performance rather than an arbitrary default.

3. Experiments

3.1 LSTM Training

The LSTM was first trained on the 80/20 split for 8 epochs. The training loss steadily decreased across all epochs, showing that the model was learning:

Epoch	Loss
1	0.3736
2	0.2169
3	0.1673
4	0.1406
5	0.1018
6	0.0757
7	0.0538
8	0.0433

On the validation split, the best threshold found was **0.846** with an F1-score of **0.7912**. The validation ROC-AUC was **0.9613**.

After threshold selection, the model was retrained on the full dataset (all epochs, more data). This slightly shifted the optimal threshold since the model saw more training examples.

3.2 TF-IDF + MLP Training

For the TF-IDF model, training was done with early stopping. The training stopped at iteration 6 because the validation score did not improve by more than 0.0001 for 3 consecutive rounds. Loss values for the first few iterations:

Iteration	Loss	Val Score
1	0.3927	0.9179
2	0.1284	0.9227
3	0.0564	0.9227
4	0.0322	0.9193
5	0.0217	0.9173
6	0.0161	0.9164

The best threshold found on validation was **0.7524** with a validation F1 of **0.7533** and a ROC-AUC of **0.9560**.

Then the model was retrained on the full undersampled dataset and tested on the official test set.

4. Analysis

4.1 Official Test Set Results

Both models were evaluated on the official test set containing 63,978 comments (57,735 non-toxic, 6,243 toxic).

LSTM Results (Threshold = 0.8460):

Metric	Class 0 (Non-Toxic)	Class 1 (Toxic)
Precision	0.98	0.53
Recall	0.92	0.83
F1-Score	0.95	0.65
Overall Accuracy	91%	
ROC-AUC	0.9550	

Confusion Matrix:

- True Negatives: 53,205
- False Positives: 4,530
- False Negatives: 1,038
- True Positives: 5,205

TF-IDF + MLP Results (Threshold = 0.7524):

Metric	Class 0 (Non-Toxic)	Class 1 (Toxic)
Precision	0.98	0.56
Recall	0.93	0.78
F1-Score	0.95	0.65
Overall Accuracy	92%	
ROC-AUC	0.9505	

Confusion Matrix:

- True Negatives: 53,878
- False Positives: 3,857
- False Negatives: 1,344
- True Positives: 4,899

4.2 Comparison Between the Two Models

Both models ended up with the same F1-score of 0.65 for the toxic class. But there are some clear differences when you look closer:

- The **LSTM** has better recall (0.83 vs 0.78), meaning it catches more actual toxic comments. It misses fewer genuinely toxic ones.
- The **TF-IDF model** has slightly better precision (0.56 vs 0.53), meaning when it says something is toxic, it is more likely to be right. It also produces fewer false positives.
- The **LSTM** has a higher ROC-AUC (0.9550 vs 0.9505), which means it has better overall discrimination ability across all thresholds.

In summary, the LSTM is a bit better at finding toxic comments (fewer missed ones), while the TF-IDF model is a bit more careful (fewer wrong flags). For a moderation use case, catching more toxic comments (high recall) is generally more important, so the LSTM has a slight edge.

4.3 Precision-Recall Trade-off

Because the dataset is heavily imbalanced (about 9:1 ratio of non-toxic to toxic), the models were deliberately tuned to favor recall over precision. A lower precision means that some clean comments will get flagged incorrectly (false positives), but a higher recall means fewer genuinely toxic comments slip through (fewer false negatives).

For an automated content moderation system, missing a highly toxic comment is considered much worse than incorrectly flagging a benign comment. The flagged comment can still go to a human reviewer, but a missed toxic comment stays visible to the public. This is why we prioritized recall during threshold selection.

4.4 Error Analysis

After testing both models, we looked at the types of mistakes each one makes.

TF-IDF Model (False Positives):

The TF-IDF model heavily relies on specific words being present or absent. If a comment contains words like "kill" or "idiot" even in an academic or neutral context (like discussing a news article), the model often flags it as toxic. This happens because TF-IDF has no understanding of context, only of which words appear and how often. A sentence like "The movie villain wants to kill everyone" could easily be misclassified.

LSTM Model (False Negatives):

The LSTM struggles with text that contains obfuscated or misspelled toxic words. For example, if someone writes "f*ck" or "sh1t" to bypass filters, the tokenizer does not recognize these as known words and maps them to an unknown token. Without the actual word embedding, the LSTM does not have enough signal to identify the comment as toxic and ends up clearing it. This is a known limitation of fixed-vocabulary models.

These two failure modes are in some ways opposite to each other: one is too sensitive to certain words, the other is too dependent on knowing the exact word.

5. Contribution

5.1 What Was Built

This project built two complete, working pipelines for toxic comment detection from scratch:

Pipeline 1: TF-IDF + MLP

- Text preprocessing with cleaning and undersampling
- TF-IDF vectorization with up to 50,000 features (unigrams + bigrams)
- MLPClassifier with early stopping
- Threshold tuning using precision-recall curve on validation data

Pipeline 2: Bidirectional LSTM

- Custom vocabulary building and word-to-index mapping stored in `word2idx.json`
- Embedding layer trained from scratch
- Bidirectional LSTM with dropout for regularization
- Two-phase training: threshold tuning on split, then full retraining
- Model saved as `toxic_lstm (1).pt` for later inference

5.2 FastAPI Inference API

Beyond just training the models, a working REST API was built using **FastAPI** to serve real-time predictions. The API loads the trained LSTM model on startup and exposes a **/predict** endpoint.

A user can send a POST request with a comment text, and the API responds with:

- Whether the comment is toxic or not (**is_toxic**: true/false)
- A confidence score between 0 and 1

Example request:

```
{
  "text": "You are an idiot and I hope you get banned."
}
```

Example response:

```
{
  "text": "You are an idiot and I hope you get banned.",
  "is_toxic": true,
  "confidence": 0.9854
}
```

The API also automatically generates an interactive documentation page (Swagger UI) accessible at **http://localhost:8005/docs**, which makes it easy to test without writing any extra code.

The preprocessing done inside the API is consistent with what was done during training: lowercasing, URL removal, number replacement, repeated character reduction, and punctuation removal. This ensures that the model gets input in the same format it was trained on.

5.3 Key Takeaways

- Both models achieved a ROC-AUC above 0.95, showing that it is possible to build strong toxic comment detectors even with relatively simple approaches like TF-IDF.
- The Bidirectional LSTM performed slightly better on recall, which is the more important metric for content moderation.
- Threshold tuning is a simple but very effective way to improve practical performance on imbalanced datasets.
- Each model has its own weaknesses. The TF-IDF model is fooled by context, and the LSTM is fooled by obfuscation. A combined ensemble approach could potentially address both.
- Deploying the model as a REST API makes it easy to integrate into any platform that wants to use automatic moderation.

References

-
- Jigsaw Toxic Comment Classification Challenge, Kaggle. <https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge>
 - PyTorch Documentation. <https://pytorch.org/docs>
 - Scikit-Learn Documentation. <https://scikit-learn.org/stable>

- FastAPI Documentation. <https://fastapi.tiangolo.com>